DATA 612 · Statistical Programming in R · Week 4

4.1 dplyr: Rows

filter(), slice(), and arrange() choose and order rows. Use nycflights13::flights.

1. Why dplyr

Real tables are messy: missing values, extra rows, names that do not match, numbers stored as text. `dplyr` is the tidyverse grammar for one table. The verbs have the same first argument, a data frame, so they pipe.

```
library(tidyverse)
library(nycflights13)
data("flights")
```

Install `nycflights13` once in the console. `flights` is every flight out of New York City in 2013. `?flights`.

A data frame is a list of atomic vectors of the same length: columns can differ in type; rows are the observational units. A **tibble** is the tidyverse data frame. We use the two words as synonyms.

The usual verbs, by what they touch:

- **rows**: `filter()`, `slice()`, `arrange()`
- **columns**: `select()`, `rename()`, `mutate()`, `relocate()`
- **groups**: `group_by()`, `summarize()`

A first pipeline. Average scheduled departure time, by carrier, in the second half of the year:

```
flights |>
  filter(month >= 7) |>
  group_by(carrier) |>
  summarize(mean_dep = mean(dep_time, na.rm = TRUE))
```

The Base R version is `flights[flights$month >= 7, ]` then `aggregate()`. The pipe is easier to read and to debug: comment out a line, or insert `View()`.

---

2. `filter()` keeps rows by value

`filter()` keeps rows where a condition is `TRUE`. Note 2.1 already compared it to `subset()`.

```
flights |>
  filter(month == 1)
```

Several conditions separated by commas are **and**. Write `&` and `|` when the logic is mixed. `xor()` is exclusive or.

```
flights |>
  filter(month == 1, origin == "JFK")

flights |>
  filter((month == 1 & origin == "LGA") | (month == 12 & origin == "JFK"))
```

Unknown levels: `unique(flights$origin)`, or `levels(as.factor(flights$origin))`.

`filter()` drops rows where the condition is `NA`. Ask for them with `is.na()`. `NA == NA` is `NA`, not `TRUE`.

```
dfdat <- data.frame(x = c(1, NA, 2), y = c(2, 4, 1))
dfdat |> filter(x == 1)
dfdat |> filter(x == 1 | is.na(x))
```

For doubles, `==` is strict. Use `dplyr::near()`:

```
sqrt(2)^2 == 2
near(sqrt(2)^2, 2)
```

Integers can use `==`.

---

3. `slice()` keeps rows by position

```
flights |> slice(5:10)
flights |> slice(c(1, 4, 6))
flights |> slice(10:n())     # n() is the number of rows
```

Helpers:

- `slice_head(n = 3)`, `slice_tail(n = 3)`
- `slice_sample(n = 200)` or `slice_sample(prop = 0.1)`
- `slice_min(arr_delay, n = 5)`, `slice_max(arr_delay, n = 5)`

Drop `NA`s in the ordering column first. Ties can return more rows than `n`. `with_ties = FALSE` cuts at `n` using the current row order.

```
flights |>
  filter(!is.na(arr_delay)) |>
  slice_max(arr_delay, n = 5)
```

`set.seed()` before `slice_sample()` if you want a repeatable draw. Two samples of the "same" size (`prop = 0.01` vs `n = 3367`) are still two draws.

---

4. `arrange()` sorts rows

Default is ascending. `desc(x)` or `-x` (numerics) reverses. Extra arguments break ties. Missing values go last, even under `desc()`.

```
flights |> arrange(dep_delay)
flights |> arrange(desc(dep_delay), arr_delay)
```

Character order follows the locale.

---

5. Practice

1. Flights in odd months, or on even days of even months (`%%`).
2. Flights that went to Houston (`IAH` or `HOU`), on UA, AA, or DL, in July–September, arrived more than 15 minutes late, and did not leave late.
3. Rows 10, 100, 1000, 10000, and 100000.
4. The last 10 of the first 30 Newark (`EWR`) flights.
5. Longest `distance`, ties broken by `air_time`.